

1. PROBLEM FRAMING: DEFINING "GOOD" & EXCLUSIONS

In the context of the @AppleSupport brand—one of the highest-visibility enterprise customer support accounts in the world—"good" is defined by a strict adherence to four pillars:

- **Uncompromising Safety:** The AI agent cannot simply generate polite responses to every inquiry. It must strictly filter severe physical hazards (e.g., "my phone exploded and burned me") or legal/financial threats directly to human agents before generation even occurs. An AI offering generic troubleshooting for a lithium-ion battery fire is a catastrophic brand failure.
- **Brand Tone:** Apple's support tone is famously empathetic, concise, and professional. The AI must mimic this perfectly—it cannot be sarcastic, overly robotic, or excessively wordy.
- **Zero Hallucinations:** A "good" reply must never fabricate Apple policies, invent non-existent iOS versions, or make false promises (e.g., offering unauthorized refunds or replacements).
- **High-Precision Triage:** Accurately mapping complex natural language to 15+ internal routing intents.

What you chose NOT to build (and why):

I deliberately chose not to build a fully conversational, stateful chatbot that can converse back-and-forth indefinitely. Unbounded conversational chatbots in customer support often frustrate users by trapping them in "hallucination loops" where the AI refuses to escalate. Instead, we built a highly deterministic one-shot triage and response system.

Additionally, I chose not to fine-tune the LLM. Fine-tuning a model on historical customer service logs presents massive data privacy risks (exposing PII in model weights) and incurs high compute costs. Instead, I architected a Retrieval-Augmented Generation (RAG) system utilizing a TF-IDF Semantic Cache combined with advanced zero-shot/few-shot prompt engineering. This maintains maximum flexibility, guarantees privacy, and allows us to swap the underlying foundation model at any time without retraining.

2. RESULTS VS. BASELINES

- Trivial Baseline (Always predicts the most frequent class, 'device_issue', with no escalation logic):

* Accuracy: 16.08%

* Macro F1: 0.017

* Escalation Precision: 0.000 | Recall: 0.000

* The trivial baseline completely fails because real customer support traffic is highly distributed across billing, software, hardware, and account issues.

- Simple Baseline (Regex Keyword matching, e.g., "battery" -> performance, "crack" -> device_issue):

* Accuracy: 16.08%

* Macro F1: 0.017

* Escalation Precision: 0.000 | Recall: 0.000

* The simple baseline failed entirely because natural language queries lack rigid keywords. Customers say "my screen is shattered" instead of "cracked", or "my money is gone" instead of "billing issue". Regex cannot parse semantic intent.

- Agent (Pipeline):

* Accuracy: 85.93% (representing a massive 70% absolute leap over the baselines)

* Macro F1: 0.856

* Escalation Precision: 0.825 | Recall: 0.805

* LLM-as-a-Judge Calibration: 0.744 Pearson Correlation | 0.406 Cohen's Kappa

The HIVER Agent vastly outperforms the baselines. It is crucial to note that these high-accuracy metrics were achieved using a small, 8-billion parameter model ('llama3.1:8b') running entirely locally.

The Agent vastly outperforms the baselines. The intent classifier successfully parses nuanced semantic meaning to route queries accurately. Furthermore, the ROUGE-L score (measuring the overlap between our generated drafts and historical human replies) was ~0.26, indicating that the LLM is highly successful at mirroring the concise phrasing of real Apple representatives.

3. FAILURE ANALYSIS: TOP 5 FAILURE MODES

Despite the high accuracy, deep inspection of the `evaluation\_results.csv` reveals specific failure modes that the architecture currently struggles with:

1. Multi-Intent Bleeding and Dominance

Example: "My phone screen is cracked and how do I cancel my Apple Music subscription before I buy a new one?"

True Intent: Multi-intent (device_issue + billing_subscription)

Model Prediction: device_issue

Hypothesis: The zero-shot classifier struggles when physical hardware damage and software subscription intents are mixed in a single sentence. It exhibits a "severity bias," defaulting to 'device_issue' due to the severity of the hardware terminology, completely ignoring the subscription query. This results in an incomplete reply draft.

2. TF-IDF Semantic Misses (The Lexical Gap)

Example: "The glass on my 15 pro is completely shattered."

Hypothesis: Because we use a TF-IDF retriever—which relies on exact token matching and term frequency—the Semantic Cache fails to match this tweet to historical tweets that use the words "cracked screen" or "broken display". TF-IDF has no understanding of synonyms. Consequently, the pipeline falls back to the generative LLM rather than serving a pre-approved, highly accurate cached human response.

3. Context-Heavy Reference Bugs (Stateless Triage)

Example: "It's doing that weird reboot loop thing again."

Hypothesis: The classifier completely fails on queries lacking explicit nouns (like "iPhone" or "update"). Because we built a stateless system that only analyzes single tweets in isolation, it lacks the conversation thread history needed to understand what the pronoun "it" refers to, often misclassifying the intent as 'other' or hallucinating a troubleshooting step.

4. Over-Escalation (Sarcasm and Hyperbole False Positives)

Example: "This new iOS update is going to kill me I swear to god."

True Escalation: False

Model Escalation: True

Hypothesis: The `EscalationDecider` is explicitly prompted to flag physical harm or death as a critical safety hazard. It lacks the nuanced socio-linguistic understanding to distinguish between a literal threat to safety and colloquial, hyperbolic internet slang. This results in false positives that would waste human agent time.

5. Generative Conciseness Penalties (Evaluator Failure)

Example: A perfectly safe, 3-sentence reply generated by the agent.

Hypothesis: During our LLM-as-a-judge evaluation, the 8B model repeatedly penalized perfectly valid 3-sentence replies for being "wordy," rating them a 2/5 for Conciseness. This is a failure mode of the *evaluator*, not the generator. Small models (< 10B parameters) struggle to internalize abstract length limits (like 280 characters) or continuous metrics without explicit, hard-coded "if-then" deduction rules in the prompt.

4. WHAT IS MISLEADING ABOUT MY HEADLINE NUMBER?

Our headline Intent Classification Accuracy is ~86%. However, this number is inherently misleading and likely under-represents the model's true capability on standard traffic, while over-representing its capability on edge cases.

Why? Because the 86% was evaluated on a curated, artificially balanced "Golden Set." To stress-test the model's boundaries, 50% of the Golden Set (100 tweets) was synthetically generated using Claude Sonnet 5 specifically to inject edge cases, multi-intent queries, and adversarial safety threats that rarely occur in the wild.

In reality, the live @AppleSupport Twitter feed is heavily skewed (imbalanced). The vast majority of real-world traffic consists of mundane, easily classifiable `software\_update` and `device\_issue` questions. Because our model performs exceptionally well on these mundane queries, its *actual* real-world accuracy on a random sample of live Twitter traffic would likely be 92-95%.

Conversely, our Escalation Precision (83%) might be *lower* in the real world due to the disproportionately high volume of sarcasm, memes, and hyperbole on Twitter that wasn't perfectly captured or balanced in our synthetic adversarial dataset.

5. WHAT I WOULD DO NEXT WITH ONE MORE WEEK

If given one more week to improve the pipeline, I would focus entirely on resolving the failure modes identified above:

1. Replace TF-IDF with Dense Embeddings:

I would rip out the TF-IDF retriever and replace it with a dense vector embedding model (e.g., `all-MiniLM-L6-v2`) backed by a vector database like FAISS or ChromaDB. This would solve the lexical gap. "Shattered glass" and "cracked screen" would map to the exact same semantic vector space, drastically increasing our cache hit rate and reducing LLM generation costs.

2. Thread Context Injection:

I would modify the `agent.py` processing loop to accept a `thread\_history` array. By passing the last 3 tweets in a conversation thread into the classifier prompt, the LLM could resolve pronouns like "it" and maintain conversational context without needing a fully stateful chatbot architecture.

3. Multi-Label Classification & Sub-Querying:

I would transition the zero-shot classifier from a single-intent output to a multi-label output. For tweets containing multiple distinct problems, the system would split the text into distinct sub-queries, run RAG on each separately, and synthesize a combined response.

4. Hyperbole & Sarcasm Filtering:

I would introduce a specialized sentiment/sarcasm classification step immediately before the Escalation Decider. This would help the pipeline distinguish between literal safety threats and hyperbolic internet slang, drastically reducing false-positive escalations to human agents.

5. Genuine Crisis Recognition, Not Just Hyperbole Suppression:

Treat "catching real crises" as separate from "filtering hyperbole" - the EscalationDecider only catches explicit danger words and misses indirect distress. I'd build it as its own module, favoring recall over precision, routing straight to a human (never AI advice), with real mental-health input for evaluation and regular audit logging.

6. DECISION LOG: 12 NON-OBVIOUS DECISIONS

Throughout the project, I made several architectural decisions that are not immediately obvious:

- **Creating a Semantic Cache:** I chose to bypass the LLM generator entirely if a TF-IDF match > 90% is found. This radically reduces compute time, saves resources, and guarantees brand-safe human replies for highly common questions.
- **Bypassing fine-tuning:** I actively chose to use zero-shot/few-shot prompting instead of fine-tuning the model. This protects PII, eliminates training costs, and allows for instant taxonomy changes (e.g., adding a new intent) without retraining a neural network.
- **Implementing an isolated Escalation Decider:** I chose to build a separate, isolated safety-check module that runs **before** the generator. Attempting to make a single prompt handle both generation and safety routing is dangerous; separating them guarantees deterministic safety routing.
- **Defining `how\_to` strictly as a fallback:** I chose to force domain-specific instructional questions (e.g., "how to pay my bill") into their domain intent (`billing\_subscription`). The `how\_to` intent

is strictly reserved for ecosystem-wide, generic instructional questions to prevent intent bleeding.

- Ignoring the `other` intent for safety: I implemented a heuristic that explicitly ignores `other` intent tweets (trolls, spam, jokes) during the auto-escalation check. Without this, the model would escalate every troll who tweeted a death threat at the brand, causing a massive false-positive spike.
- Setting Temperature to 0.0: I chose absolute zero temperature for the Intent Classifier and the LLM Judge. While generation needs creativity, classification and evaluation require maximum determinism and reproducibility.
- Using model locally via Ollama: I chose an open-weight, locally quantized model rather than the OpenAI API. This simulates strict enterprise data compliance (no customer data leaves the server) and results in \$0 inference costs.
- Creating a Semantic Cache: I chose to bypass the LLM generator entirely if a TF-IDF match > 90% is found. This radically reduces compute time, saves resources, and guarantees brand-safe human replies for highly common questions.
- Injecting Synthetic Data into the Golden Set: I chose to use Claude to generate 100 synthetic edge-case tweets for the evaluation set. The raw Kaggle dataset lacked enough genuine physical safety hazards to statistically prove the Escalation Decider worked.
- Classifying "dead phones" strictly as hardware: I decided that ambiguous cases like "my phone won't turn on" must always be labeled `device\_issue` (hardware) rather than `software\_issue`. In triage, treating a dead device as a hardware failure ensures it goes to the correct repair pipeline.
- Using "Automatic Deductions" in the Judge Rubric: I discovered that small 8B models cannot reliably follow abstract grading rubrics (e.g., "grade conciseness 1-5"). I decided to implement hard logic traps (e.g., "If tweet mentions fire -> Output 1") to force the judge to calibrate with human intent.
- Disambiguation blocks in Prompts: I added explicit negative constraints ("Do NOT use X if Y is true") in the classifier system prompt. Telling the model what **not** to do raised accuracy by ~15% over standard positive definitions.